

MOE大模型对比篇

来自：AiGC面试宝典

Just do it!

2024年06月23日 11:25

- MOE大模型对比篇
- DeepSpeed-MoE
 - PAI-Megatron-Patch MoE

DeepSpeed-MoE

- Pyramid-MoE：浅层放少量 Experts，深层放更多 Experts，形成 Pyramid 形状。这样能够一定程度上减少参数量（比如减少浅层 Experts 数量等方式）
- Residual-MoE：固定选择第一个 expert，只让第二个 expert 参与 gating 选择，就能达到与 Top2 gating 相当的效果。而前者由于 Experts 选择分发部分只有 Top2 gating 的一半，所以能够达到更低 latency。

结合 Pyramid-MoE 和 Residual-MoE 产生了 PR-MoE。相对于左图标准的 MoE 模型，PR-MoE 具有更少的参数量，更高的吞吐，以及不失精度的模型效果。

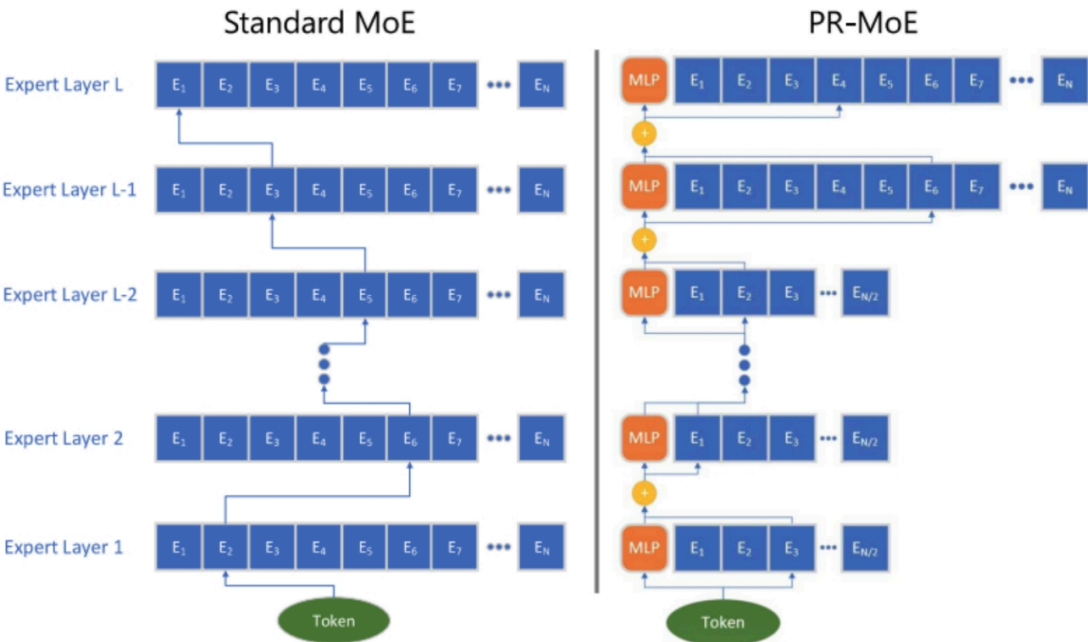


Figure 3: The illustration of standard MoE (left) and PR-MoE (right).

PAI-Megatron-Patch MoE

- Load Balancing Loss (from switch transformer)

MoE 训练中一个容易出现的问题是大多数 token 被 route 到少数几个 expert，导致少数 expert 高度特化而其他 expert 学不到东西。为了避免这一局面的出现，现在普遍加入 load balancing loss 来鼓励 router 把 token 尽可能均匀地分配到不同的 expert。

Given N experts indexed by $i = 1$ to N and a batch $\mathcal{B}$ with T tokens, the auxiliary loss is computed as the scaled dot-product between vectors f and P ,

$$\text{loss} = \alpha \cdot N \cdot \sum_{i=1}^N f_i \cdot P_i \quad (4)$$

where f_i is the fraction of tokens dispatched to expert i ,

$$f_i = \frac{1}{T} \sum_{x \in \mathcal{B}} \mathbb{1}\{\text{argmax } p(x) = i\} \quad (5)$$

and P_i is the fraction of the router probability allocated for expert i ,²

$$P_i = \frac{1}{T} \sum_{x \in \mathcal{B}} p_i(x). \quad (6)$$

- z-Loss (from ST-MoE)

模型训练不稳定因素之一是 router 在 softmax 操作前的 logits magnitude 过大，导致了大量 round-off error 的产生，这里通过 z-loss 来约束 logits 的 absolute magnitude。

$$L_z(x) = \frac{1}{B} \sum_{i=1}^B \left(\log \sum_{j=1}^N e^{x_j^{(i)}} \right)^2 \quad (5)$$

z-loss 会使得模型尽量产生数值较小的 logits，从而产生较好的模型 stability 与 quality 的 trade-off，而 clipping logits 通常会导致更大的 round-off error 和不连续性。

• Deepseek MoE

现有的 MoE 架构可能存在知识混杂（Knowledge Hybridity）和知识冗余（Knowledge Redundancy）的问题，限制了专家的专业化。

- 细粒度专家划分：不同于传统 MoE 直接从与标准 FFN 大小相同的 N 个专家里选择激活 K 个专家（如 Mistral 7B*8 采取 8 个专家选 2 专家），我们把 N 个专家粒度划分更细，在保证激活参数量不变的情况下，从 mN 个专家中选择激活 mK 个专家（如 DeepSeekMoE 16B 采取 64 个专家选 8 个专家），如此可以更加灵活地组合多个专家
- 共享专家分离：我们把激活专家区分为共享专家（Shared Expert）和独立路由专家（Routed Expert），如上图 4(c)，此举有利于将共享和通用的知识压缩进公共参数，减少独立路由专家参数之间的知识冗余

